Module 5 – Group Assignment
Reverse Engineering Architectural Decisions

System: Hadoop Distributed File System (HDFS)

Member Participation

Ulises Almonte – 100%
Thong Ho – 100%
Chetana Kamraj Patel – 100%
Suvarna Ramu – 100%

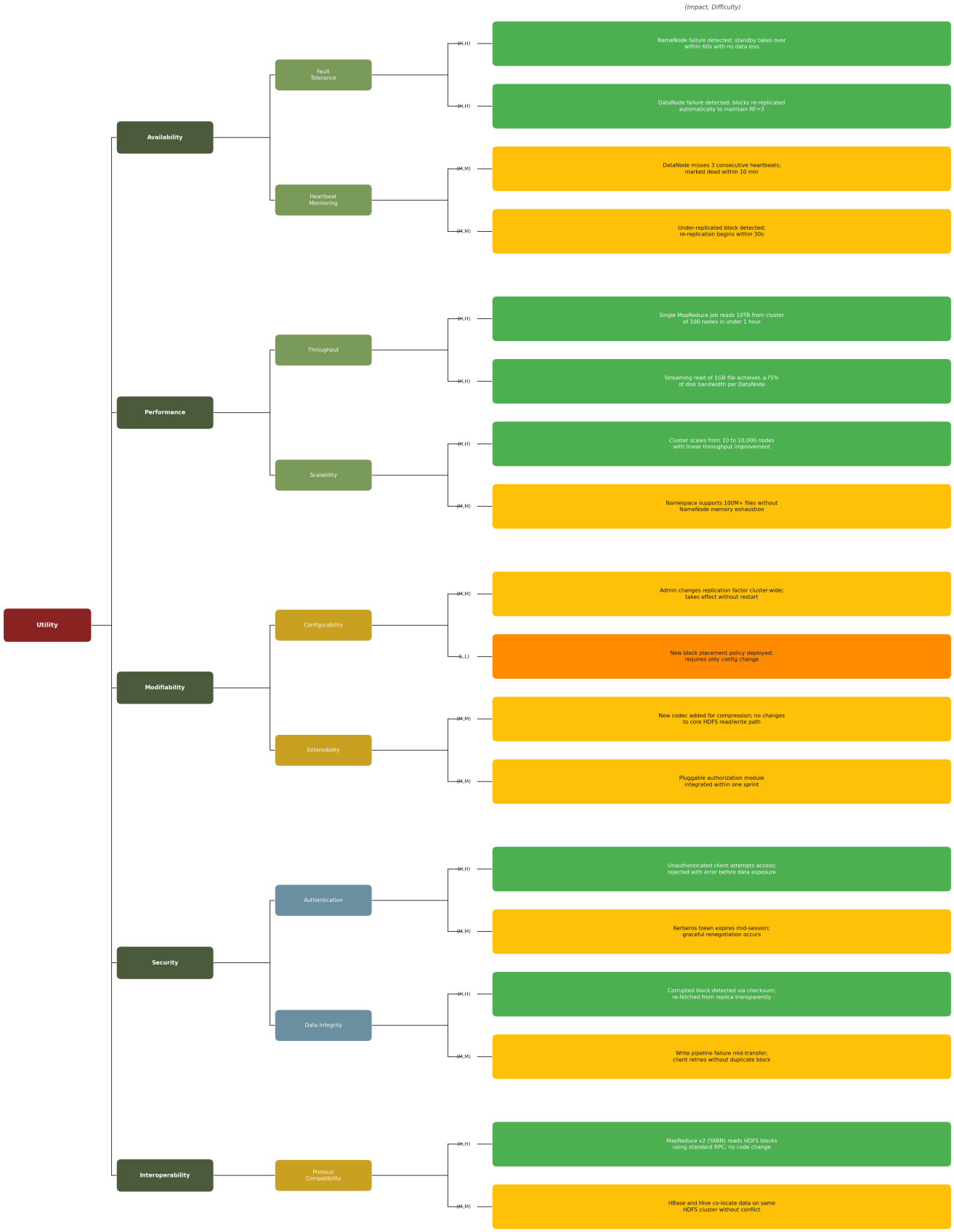
Section 1: Utility Tree

The utility tree below decomposes HDFS's overall system's utility into quality attributes, refines each attribute into specific scenarios, and assigns each scenario as a priority pair (Impact, Difficulty). Scenarios highlighted in green are high priority (H, H), yellow are medium (M, M), and orange are lower priority (L, L).

Quality Attribute	Attribute Refinement	Quality Attribute Scenarios (Priority)	
Availability	Fault Tolerance	NameNode failure detected; standby takes over within 60s with no data loss Priority: (H, H)	DataNode failure detected; blocks re-replicated automatically to maintain RF=3 Priority: (H, H)
	Heartbeat Monitoring	DataNode misses 3 consecutive heartbeats; marked dead within 10 min Priority: (M, M)	Under-replicated block detected; re-replication begins within 30s Priority: (M, M)
Performance	Throughput	Single MapReduce job reads 10TB from cluster of 100 nodes in under 1 hour Priority: (H, H)	Streaming read of 1GB file achieves ≥75% of disk bandwidth per DataNode Priority: (H, H)

	Scalability	Cluster scales from 10 to 10,000 nodes with linear throughput improvement Priority: (H, H)	Namespace supports 100M+ files without NameNode memory exhaustion Priority: (M, M)
Modifiability	Configurability	Admin changes replication factor cluster-wide; takes effect without restart Priority: (M, M)	New block placement policy deployed; requires only config change Priority: (L, L)
	Extensibility	New codec added for compression; no changes to core HDFS read/write path Priority: (M, M)	Pluggable authorization module integrated within one sprint Priority: (M, M)
Security	Authentication	Unauthenticated client attempts access; rejected with error before data exposure Priority: (H, H)	Kerberos token expires mid-session; graceful renegotiation occurs Priority: (M, M)
	Data Integrity	Corrupted block detected via checksum; re-fetched from replica transparently Priority: (H, H)	Write pipeline failure mid-transfer; client retries without duplicate block Priority: (M, M)
Interoperability	Protocol Compatibility	MapReduce v2 (YARN) reads HDFS blocks using standard RPC; no code change Priority: (H, H)	HBase and Hive co-locate data on same HDFS cluster without conflict Priority: (M, M)

Priority Key: H = High, M = Medium, L = Low | Format: (Impact, Difficulty)


(Impact, Difficulty)
Utility
Availability
Fault Tolerance
(H,H)
NameNode failure detected; standby takes over within 60s with no data loss
(H,H)
DataNode failure detected; blocks re-replicated automatically to maintain RF=3
Heartbeat Monitoring
(M,M)
DataNode misses 3 consecutive heartbeats; marked dead within 10 min
(M,M)
Under-replicated block detected; re-replication begins within 30s
Performance
Throughput
(H,H)
Single MapReduce job reads 10TB from cluster of 100 nodes in under 1 hour
(H,H)
Streaming read of 1GB file achieves ≥75% of disk bandwidth per DataNode
Scalability
(H,H)
Cluster scales from 10 to 10,000 nodes with linear throughput improvement
(M,M)
Namespace supports 100M+ files without NameNode memory exhaustion
Modifiability
Configurability
(M,M)
Admin changes replication factor cluster-wide; takes effect without restart
(L,L)
New block placement policy deployed; requires only config change
Extensibility
(M,M)
New codec added for compression; no changes to core HDFS read/write path
(M,M)
Pluggable authorization module integrated within one sprint
Security
Authentication
(H,H)
Unauthenticated client attempts access; rejected with error before data exposure
(M,M)
Kerberos token expires mid-session; graceful renegotiation occurs
Data Integrity
(H,H)
Corrupted block detected via checksum; re-fetched from replica transparently
(M,M)
Write pipeline failure mid-transfer; client retries without duplicate block
Interoperability
Protocol Compatibility
(H,H)
MapReduce v2 (YARN) reads HDFS blocks using standard RPC; no code change
(M,M)
HBase and Hive co-locate data on same HDFS cluster without conflict

Decision #1: Master–Worker Architecture (NameNode + DataNodes)

What is the decision and what is the rationale for this decision?

HDFS adopts a single-master (NameNode) and multiple-worker (DataNode) architecture in which the NameNode manages filesystem metadata namespace, permissions, and block locations while DataNodes store and transfer the actual file blocks. Clients first contact the NameNode to locate blocks and then communicate directly with DataNodes for data transfer.

The key architectural decision is the centralization of metadata combined with distributed data transfer. An alternative design would have been a fully decentralized peer-to-peer filesystem in which every node tracks metadata, or a fully centralized storage server that handles both metadata and data. A decentralized approach would create excessive coordination overhead and slow file lookup at large scale, while a fully centralized server would become a throughput bottleneck as the cluster grows. HDFS instead separates control and data paths: the NameNode provides authoritative coordination and consistency, while bulk data movement occurs directly between clients and DataNodes. When a client reads a 1TB log file, it contacts the NameNode once to retrieve block locations, then streams all data in parallel directly from the relevant DataNodes the NameNode bears none of that transfer load, which is what allows it to serve thousands of concurrent clients without degrading performance.

The target workload drives the rationale for very large files accessed in streaming fashion on unreliable commodity hardware. Centralized metadata allows fast failure detection and consistent decision-making, while distributed block transfer preserves high throughput across the cluster.

What architecturally significant requirements were goals that this decision helped satisfy?

This decision directly supports several high-priority utility tree scenarios across multiple quality attributes. For availability, the NameNode's continuous tracking of block placement and DataNode heartbeats enables two (H,H) scenarios: DataNode failure detected; blocks re-replicated automatically to maintain RF=3, and NameNode failure detected; standby takes over within 60 seconds with no data loss. When a DataNode stops responding, the NameNode identifies which blocks have dropped below their replication factor and schedules re-replication on a surviving node automatically a recovery process that requires no administrator intervention because a single coordinator holds the complete view of cluster state at all times.

For performance, the direct client-to-DataNode transfer model satisfies the throughput scenarios: single MapReduce job reads 10TB from 100 nodes in under one hour (H,H), and streaming reads achieve at least 75% of disk bandwidth per DataNode (H,H). Since the NameNode only answers location queries and does not carry data traffic, all 100 nodes can stream simultaneously without any central bottleneck. For scalability, adding DataNodes increases storage and bandwidth without adding load to the NameNode, directly satisfying cluster scales from 10 to 10,000 nodes with linear throughput improvement (H,H). For data integrity, the NameNode's global block inventory allows it to redirect a client from a corrupt block to a healthy replica, satisfying corrupted block detected via checksum; re-fetched from replica transparently (H,H).

Why is this decision Important?

This architectural decision allows HDFS to satisfy its most critical requirements simultaneously, availability, scalability, and throughput. If metadata and data transfer were both centralized, performance would fail under large workloads. If metadata were fully decentralized, failure detection and recovery guarantees would fail. The chosen design accepts the tradeoff of a logically centralized coordinator in exchange for predictable recovery behavior and simplified consistency management.

The NameNode/DataNode separation also shaped every downstream design element in the replication pipeline, rack-aware placement, lease management, and journaling all assume a single metadata authority. Because the NameNode has a complete view of cluster topology and block state at all times, it can detect under-replication, enforce placement policy, and coordinate failover without inter-node negotiation. The architects later added NameNode High Availability a standby node with a shared edit log to address the single point of failure. The fact that this required no changes to the data path confirms that the original separation was sound. The master-worker decision did not just solve an immediate design problem; it established boundaries that kept the system modifiable as requirements evolved, directly enabling the long-term satisfaction of the high-priority availability and scalability scenarios in the utility tree.

Decision #2: RPC-Based Communication over TCP/IP and DFSShell Interface

What is the decision and what is the rationale for this decision?

HDFS adopts a Remote Procedure Call (RPC) communication model built on TCP/IP for all client-to-NameNode and NameNode-to-DataNode interactions and provides a command-line interface

(DFSShell) modeled after UNIX filesystem semantics for structured user interaction with the distributed file system.

The architectural decision is twofold: standardized RPC communication over TCP/IP, and a filesystem like an abstraction layer through DFSShell. The rationale for choosing TCP/IP-based RPC over a proprietary protocol is rooted in portability and interoperability. TCP/IP is universally supported across commodity hardware and operating systems, allowing HDFS to operate in heterogeneous environments without custom network infrastructure. A proprietary protocol could have offered marginal latency improvements but would have increased integration complexity and reduced ecosystem compatibility an unacceptable tradeoff for a system designed to work alongside MapReduce, HBase, and Hive.

RPC further abstracts communication details from application logic. Clients invoke high-level operations such as open, read, and write rather than managing raw socket communication, which reduces coupling between distributed components and isolates internal changes from external consumers. The DFSShell interface reinforces this abstraction by presenting a logical namespace view files, directories, and permissions rather than exposing block-level storage details. This separation between storage mechanics and user interaction reflects clear architectural layering and aligns with the modifiability tactics covered in this course.

What architecturally significant requirements were goals that this decision helped satisfy?

This decision directly supports several high-priority utility tree scenarios across security, interoperability, and performance. For security, RPC communication flows through controlled ports on the NameNode and DataNodes, allowing administrators to enforce network-level access policies. Combined with Kerberos integration, this ensures unauthorized clients are rejected before any data is exposed, directly satisfying the (H,H) scenario: unauthenticated client attempts access, rejected with error before data exposure. Without controlled RPC endpoints, enforcing consistent authentication across a cluster of thousands of nodes would require fragmented, node-level policy management with no central enforcement point.

For interoperability, standardized TCP/IP RPC allows external systems MapReduce v2 (YARN), HBase, and Hive to communicate with HDFS without custom protocol adapters, directly satisfying the (H,H) scenario: MapReduce v2 (YARN) reads HDFS blocks using standard RPC; no code change required. This also satisfies the (M,M) scenario: HBase and Hive co-locate data on the same HDFS cluster without conflict, since all frameworks speak the same protocol to the same endpoints. For performance, RPC handles control-plane traffic metadata lookups and block location queries while bulk data transfer occurs directly between clients and DataNodes over raw TCP streams, keeping the NameNode off the data path and supporting the high-throughput streaming scenarios in the utility tree. For modifiability, abstracting communication behind RPC

interfaces means internal changes to metadata handling, replication logic, or authorization can occur without modifying client interaction models, satisfying the pluggable authorization module (M,M) and extensibility scenarios.

Why is this decision important?

This decision establishes the communication backbone of HDFS and is architecturally important because it enables the system to satisfy security, interoperability, and modifiability requirements simultaneously through a single structural choice rather than separate mechanisms. Without standardized RPC over TCP/IP, integration with YARN, Hive, and HBase would require custom adapters for each framework, security enforcement would become fragmented across nodes, and future protocol changes would risk breaking ecosystem compatibility across the entire Hadoop stack.

The decision also reflects deliberate tradeoffs consistent with HDFS's architectural drivers. TCP/IP increases portability but does not achieve the absolute minimum latency of a specialized binary protocol. RPC abstraction improves modularity but introduces serialization overhead. The DFSShell filesystem abstraction simplifies administration but hides block-level optimization opportunities from users. Each tradeoff was accepted because HDFS was designed for commodity hardware, heterogeneous environments, and ecosystem integration not for latency-sensitive workloads where a proprietary protocol would be justified. Had HDFS adopted a proprietary binary protocol optimized solely for throughput, it might have marginally reduced latency but at the cost of the interoperability and modifiability scenarios that the utility tree identifies as architecturally significant. The RPC and DFSShell decision ensures those scenarios remain satisfiable as the system evolves.

Decision #3: Rack Aware Placement and Replication Policy

What is the decision and what is the rationale for this decision?

In the Hadoop Distributed File System (HDFS), DataNodes are grouped into racks connected by the same network switch, meaning all nodes within a rack share the same single point of failure. By default, each file block has three replicas distributed according to the rack-aware placement policy. When a file is written, the policy places the first replica on the node where the client is writing, leveraging data locality to improve read and write performance. The second replica is placed on a node in a different rack entirely, ensuring cross-rack redundancy. The third replica is placed on a different node within that same second rack, keeping intra-rack copy cost low since nodes on the same rack share a high bandwidth switch rather than traversing inter-rack network links.

This placement reflects a deliberate tradeoff. Placing all three replicas on separate racks would maximize redundancy, but would triple inter-rack write traffic, which is typically the most constrained network resource in large clusters. Placing all replicas on the same rack would eliminate rack-level fault tolerance entirely. The chosen policy achieves rack-level redundancy with only one cross-rack hop per write. The replication pipeline reinforces this: the client writes only to the first DataNode, which forwards to the second, which forwards to the third, meaning the client bears a single network hop regardless of replication factor and write throughput is preserved.

What architecturally significant requirements were goals that this decision helped satisfy?

This decision directly satisfies several high and medium priority utility tree scenarios across availability, performance, and scalability. For fault tolerance, distributing replicas across racks ensures that a complete rack failure such as a top-of-rack switch outage never destroys all copies of a block, directly satisfying DataNode failure detected; blocks re-replicated automatically to maintain RF=3 (H,H). When any DataNode goes offline, the NameNode detects the under-replication condition and schedules a new replica using the same rack-aware policy, restoring the replication factor without administrator intervention and supporting the standby NameNode takeover scenario (H,H) by ensuring data remains accessible even during significant infrastructure events.

For performance, placing the first replica on the client's local node means MapReduce task schedulers can assign compute tasks to the machine where the data already resides, eliminating network transfer for the majority of reads and directly satisfying the throughput scenario: single MapReduce job reads 10TB from 100 nodes in under one hour (H,H). For scalability, as new racks are added to the cluster the placement algorithm naturally distributes replicas across them, preventing any single rack from becoming a bottleneck and reinforcing cluster scales from 10 to 10,000 nodes with linear throughput improvement (H,H). For modifiability, the replication factor is configurable per file and per directory, satisfying admin changes replication factor cluster-wide; takes effect without restart (M,M).

Why is this decision important?

The rack-aware replication policy is important because it resolves a conflict between availability and performance through a single topology-aware design decision rather than requiring separate mechanisms for each. A naive random placement policy would satisfy fault tolerance on average but would frequently place all three replicas on the same rack, leaving them vulnerable to a single infrastructure failure. The chosen policy avoids this while containing cross-rack to write traffic to one hop, satisfying both the availability and throughput scenarios without architectural compromise.

This decision is also important because the placement policy is implemented as a pluggable interface in the NameNode, meaning organizations with custom topologies such as multi-zone cloud deployments can substitute their own placement logic without modifying core HDFS code, directly satisfying the extensibility scenario, new block placement policy deployed requires only a config change (L,L). The rack-aware policy is therefore not just a fault tolerance mechanism it is a structural decision that keeps the system adaptable across diverse deployment environments while maintaining the durability and read locality guarantees that the high-priority utility tree scenarios depend on.